先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 08 – Linear Models for Regression and Classification

Chizhi Chris ZHANG

zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Three periods: fit values, fit probabilities, evaluate decisions

45 minutes	Linear regression: design matrix, normal equations, worked fit, regularization, gradient descent, and residuals.
45 minutes	Regression metrics and calculation exercise (22.5); logistic probabilities, likelihood, gradient, and softmax (22.5).
45 minutes	Classification metrics and ranking (22.5); imbalance, sampling, prevalence, and cost/resource exercise (22.5).

Connected targets

Predict a continuous examination score, then predict the probability of meeting a standard.

Outcome

Compute a fit, explain a parameter update, and evaluate the costly kind of mistake.

The same inputs can support two different prediction tasks

Question	Regression	Classification
Target	A continuous score $y \in \mathbb{R}$	Meeting a standard $y \in \{0, 1\}$
Model output	Predicted score $\hat{y}$	Probability $p(y = 1 \mid x)$
Typical loss	Squared residual	Negative Bernoulli log likelihood
Decision	Report a value	Compare probability with a chosen threshold

Both models use $z = b + w^\top x$. Linear regression reports z directly; logistic regression reports $\sigma(z)$. The target type determines how the score is interpreted and evaluated.

A constant baseline is fitted by averaging the responses

Before using a feature, minimize squared error with a single constant prediction a on the running responses $(2, 3, 5)$.

$$J(a) = \sum_i (a - y_i)^2, \quad J'(a) = 2\sum_i (a - y_i) = 0 \Rightarrow a = 10/3.$$

- The baseline RSS is $14/3$. It provides a reference that every more elaborate fit should justify improving.
- The mean is computed on training responses and reused for new predictions; it is not refitted using test labels.
- An intercept-only model is still a learned predictor. It isolates the contribution of the input feature from simply knowing the response level.

A design matrix puts all sample equations together

Use an illustrative data set $x = (1, 2, 3)$ and $y = (2, 3, 5)$. Fit $\hat{y}_i = b + wx_i$.

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad \beta = \begin{bmatrix} b \\ w \end{bmatrix}, \quad y = \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}, \qquad \hat{y} = X\beta.$$

- Rows are observations; columns are the constant and the measured feature.
- X has shape $n \times (d + 1)$; β has $d + 1$ entries.
- The intercept allows a reference level independent of the slope. Removing it forces the line through the origin.

Adding a squared feature changes the columns, not the matrix method

For the same inputs $(1, 2, 3)$, add the known feature x^2 to the design matrix.

$$X_{\text{quad}} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \end{bmatrix}, \qquad \beta = (2, -1/2, 1/2)^\top.$$

- The resulting prediction is $2 - x/2 + x^2/2$, which exactly fits responses $2, 3, 5$.
- At input four it predicts eight, while the fitted straight line predicts $19/3$. Both extrapolate beyond observed inputs.
- Linear in parameters does not mean linear in raw input. Added columns improve training flexibility, but their unseen behavior still needs validation.

Residuals define the least-squares objective

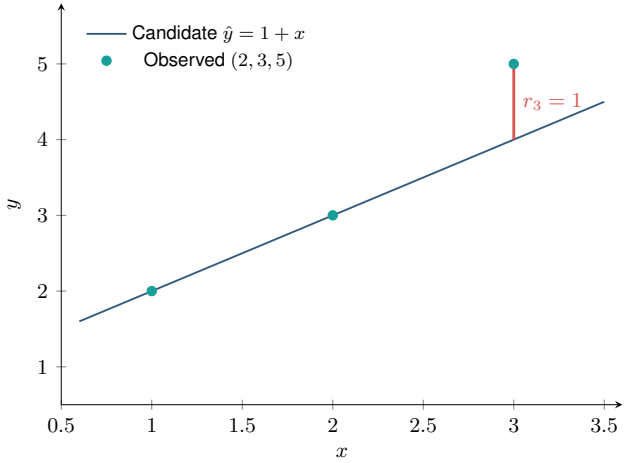
Define $r_i = y_i - \hat{y}_i$ and

$$\mathrm{RSS}(\beta) = \sum_i r_i^2 = \|y - X\beta\|_2^2.$$

For the candidate line $\hat{y} = 1 + x$, predictions are $(2, 3, 4)$; residuals are $(0, 0, 1)$, so RSS is 1.

- Positive residual means the prediction is too low; negative means too high.
- Squaring prevents cancellation and emphasizes large deviations.
- We minimize a sum for the derivation. Dividing by n leaves the unpenalized minimizer unchanged, but affects the numerical scale of a regularization coefficient.

Residuals define the least-squares objective



Compare several candidates under one residual convention

For the running observations, compare four simple prediction rules using the same squared-error sum.

$$\mathrm{RSS}(0) = 38, \quad \mathrm{RSS}(10/3) = 14/3, \quad \mathrm{RSS}(1 + x) = 1, \quad \mathrm{RSS}(1/3 + 1.5x) = 1/6.$$

- The constant baseline removes the overall response level; the feature-based lines explain additional variation.
- The OLS solution has the smallest RSS among all affine lines, not merely among these four displayed candidates.
- These are training comparisons. A high-degree rule can reduce training RSS further without establishing a better prediction model.

Differentiate the objective to obtain normal equations

$$J(\beta) = (X\beta - y)^\top (X\beta - y) = \beta^\top X^\top X\beta - 2\beta^\top X^\top y + y^\top y.$$
$$\nabla J(\beta) = 2X^\top (X\beta - y) = 0 \quad \Rightarrow \quad X^\top X\hat{\beta} = X^\top y.$$

- The objective is a convex quadratic, so a stationary solution is globally minimizing.
- A unique coefficient vector requires full column rank.
- Write the normal equations first; $(X^\top X)^{-1} X^\top y$ is valid only when that inverse exists.

The scalar derivatives explain each normal-equation row

Write the residual as $r_i = y_i - b - wx_i$ before differentiating the squared-loss sum.

$$\frac{\partial J}{\partial b} = -2\sum_i r_i, \qquad \frac{\partial J}{\partial w} = -2\sum_i x_i r_i.$$

- At an optimum with an intercept, residuals sum to zero. The slope condition also removes residual correlation with the feature column.
- On the running data, these conditions become $3b + 6w = 10$ and $6b + 14w = 23$.
- The matrix equation is a compact collection of these component conditions. Checking signs and dimensions in scalar form can prevent memorized-formula errors.

Solve the running example and check its residuals

$$X^\top X = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}, \qquad X^\top y = \begin{bmatrix} 10 \\ 23 \end{bmatrix}.$$

Solving $3b + 6w = 10$ and $6b + 14w = 23$ gives

$$w = \tfrac{3}{2}, \quad b = \tfrac{1}{3}, \qquad \hat{y} = (\tfrac{11}{6}, \tfrac{10}{3}, \tfrac{29}{6}).$$

Residuals are $(1/6, -1/3, 1/6)$, giving RSS $= 1/6$, smaller than the earlier candidate's RSS 1.

Interpretation

Within this fitted relation, a one-unit input increase adds 1.5 response units. This describes an association; the three observations do not establish causation.

The slope is response covariation divided by input variation

For one input and an intercept, center the data around their training means.

$$\hat{w} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = \frac{3}{2} = 1.5, \qquad \hat{b} = \bar{y} - \hat{w}\bar{x} = 1/3.$$

- Here $\bar{x} = 2$ and $\bar{y} = 10/3$. Positive covariation produces a positive fitted slope.
- If the centered input has zero squared norm, all inputs are equal and this slope is not identifiable.
- The formula describes a fitted association. It does not establish that changing the input causes the estimated response change.

At the optimum, residuals are orthogonal to fitted columns

From the normal equations,

$$X^\top r = 0.$$

For the running residual vector $(1/6, -1/3, 1/6)$,

$$\sum_i r_i = 0, \qquad \sum_i x_i r_i = 1/6 - 2/3 + 1/2 = 0.$$

- The fitted response is the orthogonal projection of y onto the column space of X.
- No additional movement in an available column direction can lower the squared residual norm.
- Mean training residual zero follows when an intercept is included; it is not a claim that future errors have mean zero.

Orthogonal projection gives a check on the fitted vector length

The response vector is the fitted vector plus an orthogonal residual vector.

$$y = \hat{y} + r, \quad \hat{y}^\top r = 0 \Rightarrow \|y\|^2 = \|\hat{y}\|^2 + \|r\|^2 = 227/6 + 1/6 = 38.$$

- The running fitted values are $11/6, 10/3, 29/6$, whose squared norm is $227/6$.
- The residual norm squared is RSS $1/6$. Orthogonality removes the cross term when expanding the sum.
- This identity checks the least-squares geometry in observation space. It is not a decomposition of future prediction error into bias and variance.

Rank deficiency and small singular values need different care

- If a feature column is copied, only the sum of its two coefficients affects prediction. $X^\top X$ becomes singular; coefficients are not uniquely identified.
- With $X = U\Sigma V^\top$, a pseudoinverse solution is $\hat{\beta} = X^+ y = V\Sigma^+ U^\top y$.
- The pseudoinverse gives a minimum-norm least-squares solution when several minimizers exist.
- Very small nonzero singular values amplify noise through $1/\sigma_j$; nearly redundant features can destabilize coefficients.

Practical choice

Use a numerical least-squares solver rather than explicit inversion. Remove redundant features or compare regularized fits when coefficient stability matters.

Duplicate columns make individual coefficients nonunique

Copy the running feature x into two columns and keep the same intercept.

$$f(x) = 1/3 + w_1 x + w_2 x, \quad w_1 + w_2 = 1.5.$$

- Pairs $(1.5, 0), (0, 1.5), (0.75, 0.75)$ give exactly the same training and future predictions for these duplicated inputs.
- The pseudoinverse selects the minimum-norm split $(0.75, 0.75)$ among this equivalent set.
- A chosen numerical solution does not mean the data distinguished two separate feature effects. Identifiability concerns what can be learned from the available columns.

Small singular values amplify small response changes

In one singular direction, least squares divides the projected response a_j by the singular value s_j.

$$w_j = a_j / s_j : \quad a_j = 0.10 \to 0.11; \quad s_j = 0.01 : \ 10 \to 11; \quad s_j = 1 : \ 0.10 \to 0.11.$$

- The same response perturbation changes the weak-direction coefficient by one but the strong-direction coefficient by 0.01.
- Large compensating coefficients can arise from poorly supported directions, even when training predictions fit closely.
- Use stable solvers and compare regularization or removal of redundant columns. Numerical conditioning and the model's substantive validity are distinct concerns.

Gaussian noise turns maximum likelihood into least squares

Assume independent responses with constant noise variance:

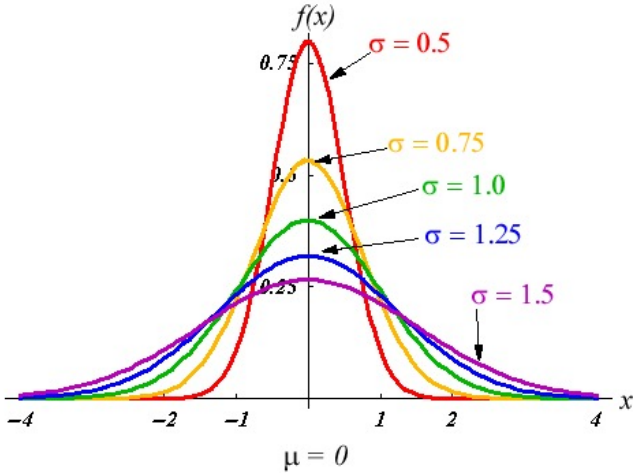
$$y_i = x_i^\top \beta + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2).$$

$$\log p(y \mid X, \beta, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_i (y_i - x_i^\top \beta)^2.$$

For fixed σ^2, only the squared-residual term changes with β. Maximizing likelihood therefore minimizes RSS.

- Least squares can be computed without Gaussian noise; Gaussianity supplies this likelihood interpretation and supports specific uncertainty formulas.
- Heterogeneous variance or unusual residual structure calls for revisiting the probabilistic model.

Gaussian noise turns maximum likelihood into least squares



Unequal noise variances lead to weighted least squares

Suppose independent measurements have known variances $1, 4, 0.25$, respectively.

$$J(\beta) = \sum_i \frac{(y_i - x_i^\top \beta)^2}{\sigma_i^2}; \qquad \text{weights} = (1, 0.25, 4).$$

- A residual of magnitude two contributes $4, 1, 16$ in the three cases; a more precise measurement receives more influence.
- The Gaussian likelihood gives these weights when the variance model is specified and parameter-independent.
- Weighting by precision is different from arbitrary preference for convenient samples. If the variances are estimated or misspecified, validate the complete procedure and its residual behavior.

Laplace noise gives an absolute-residual likelihood objective

A different conditional noise assumption produces a different training loss.

$$p(y_i \mid x_i, \beta) = \frac{1}{2b} \exp[-|y_i - x_i^\top \beta|/b] \Rightarrow -\log p(D \mid \beta) = \text{constant} + \frac{1}{b} \sum_i |r_i|.$$

- For a fixed noise scale b, maximum likelihood minimizes absolute residuals rather than squared residuals.
- Large misses grow linearly in this objective, so one extreme observation has less relative influence.
- The loss is continuous but has a nondifferentiable point at zero; appropriate optimization methods can still solve it. The noise assumption supplies an interpretation, not proof of fit quality.

An outlier changes a squared-loss fit more than a small miss

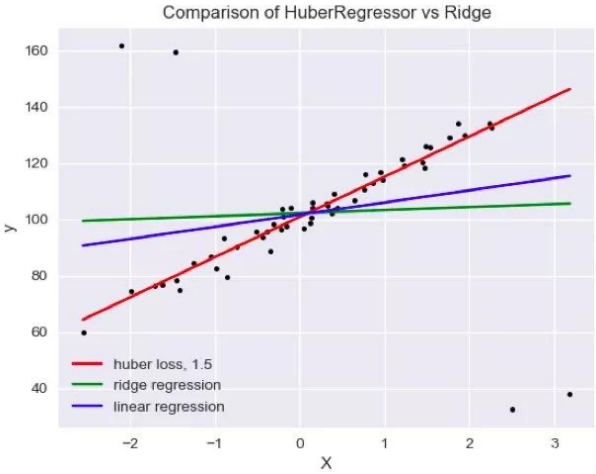
For residual r, compare losses:

$$L_2(r) = \tfrac{1}{2}r^2, \quad L_1(r) = |r|, \quad L_\delta(r) = \begin{cases} \tfrac{1}{2}r^2, & |r| \le \delta, \\ \delta|r| - \tfrac{1}{2}\delta^2, & |r| > \delta. \end{cases}$$

With $\delta = 1$, residuals 1 and 10 have Huber losses 0.5 and 9.5, compared with squared losses 0.5 and 50.

- Huber is quadratic near zero and linear for large residuals.
- L1 is continuous but nondifferentiable at zero.
- Investigate suspect observations before removing them. A rare but valid case is different from a recording error.

An outlier changes a squared-loss fit more than a small miss



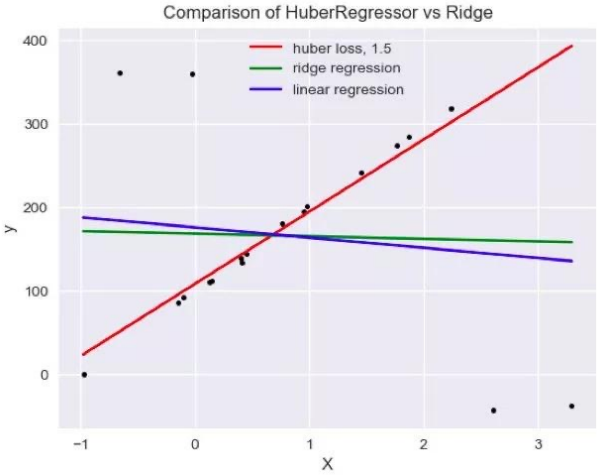
Huber caps the residual force instead of ignoring every outlier

For Huber loss with $\delta = 1$, examine its derivative with respect to a residual.

$$L'_\delta(r) = \begin{cases} r, & |r| \le 1, \\ \operatorname{sign}(r), & |r| > 1; \end{cases} \qquad r = 0.5 \Rightarrow 0.5, \quad r = 5 \Rightarrow 1.$$

- The squared-loss derivative would be 0.5 and 5 under the matching half-squared convention.
- Small residuals still receive smooth proportional correction; large residuals no longer provide an arbitrarily large pull.
- Huber does not identify which observation is a recording error. Investigate data provenance and select the transition scale using an appropriate validation design.

Huber caps the residual force instead of ignoring every outlier



A linear model can use nonlinear features and category codes

$$\phi(x) = (1, x, x^2), \qquad f_\beta(x) = \beta_0 + \beta_1 x + \beta_2 x^2.$$

The curve is nonlinear in x but linear in the coefficients. Least-squares machinery still applies to the transformed design matrix.

- An interaction $x_1 x_2$ models a feature effect that depends on another feature.
- For three categories and an intercept, use two indicator columns with a reference category; three indicators plus the intercept are redundant.
- Added columns increase flexibility and can create instability. Select useful expansions by validation, with preprocessing fitted inside folds.

A reference category avoids redundant indicator columns

Suppose a feature has categories A, B, and C, and the model includes an intercept.

$$\phi(A) = (1, 0, 0), \quad \phi(B) = (1, 1, 0), \quad \phi(C) = (1, 0, 1).$$

- The intercept is the A reference level; the next two coefficients describe differences from A.
- If all three category indicators are included with an intercept, their sum equals the constant column, making the design rank deficient.
- Category numbers 1, 2, 3 would impose an unjustified ordering and equal spacing unless the feature really has that meaning. Encoding is part of model specification, not cosmetic formatting.

Ridge adds a positive term to the weak directions

Keep the intercept separate; use centered features Z and response y_c.

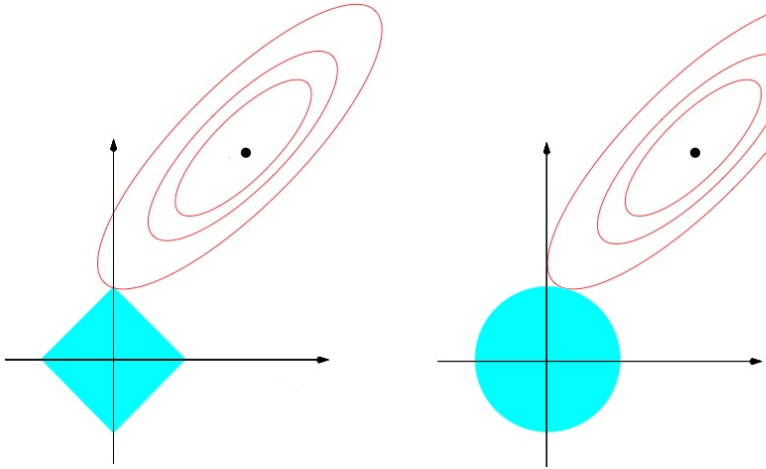
$$J(w) = \|Zw - y_c\|^2 + \lambda\|w\|^2, \qquad \hat{w} = (Z^\top Z + \lambda I)^{-1} Z^\top y_c.$$

In the running example, $Z = (-1, 0, 1)^\top$, so $Z^\top Z = 2$ and $Z^\top y_c = 3$.

$$\lambda = 1: \quad \hat{w} = 3/(2+1) = 1, \quad \hat{b} = \bar{y} - \hat{w}\bar{x} = 4/3.$$

The slope falls from 1.5 to 1. This reduces sensitivity while increasing training RSS. If loss is averaged instead of summed, the corresponding matrix uses $n\lambda I$ under that convention.

Ridge adds a positive term to the weak directions



Check the ridge solution by comparing the penalized objectives

At $\lambda = 1$ with an unpenalized intercept, the running ridge fit is $\hat{y} = 4/3 + x$.

$$\mathrm{RSS}_{\mathrm{ridge}} = 2/3, \quad J_{\mathrm{ridge}} = 2/3 + 1 = 5/3; \quad J_{\mathrm{OLS}} = 1/6 + (1.5)^2 = 29/12.$$

- Ridge has larger training RSS than OLS, but a smaller objective after the slope penalty is included.
- Its predictions are $7/3, 10/3, 13/3$; residuals are $-1/3, -1/3, 2/3$.
- This verifies why a training-fit sacrifice is intentional. Compare held-out prediction losses, excluding the parameter penalty, to decide whether the sacrifice helps the target use.

Lasso uses a thresholded coordinate update

For a centered column z_j, hold other coefficients fixed and define partial residual $r^{(-j)} = y_c - \sum_{k \neq j} z_k w_k$. With

$$J = \tfrac{1}{2}\|Zw - y_c\|^2 + \lambda \sum_j |w_j|,$$

coordinate descent gives

$$w_j \leftarrow \frac{S(z_j^\top r^{(-j)}, \lambda)}{z_j^\top z_j}, \qquad S(a, \lambda) = \operatorname{sign}(a)\max(|a| - \lambda, 0).$$

Running example: numerator evidence is 3, denominator 2; $\lambda = 1$ gives $w = 1$, while $\lambda = 4$ gives $w = 0$. The threshold can remove a coefficient exactly; correlated features can make the selected set unstable.

The zero-coefficient condition comes from the subgradient interval

For the centered one-feature Lasso objective $\frac{1}{2}\|zw - y_c\|^2 + \lambda|w|$, inspect stationarity at zero.

$$0 \in -z^\top y_c + \lambda[-1, 1] \quad \Longleftrightarrow \quad |z^\top y_c| \leq \lambda.$$

- In the running data, $z^\top y_c = 3$. Any $\lambda \geq 3$ permits the slope to be zero.
- The unpenalized intercept then remains the response mean $10/3$; the model becomes a constant baseline.
- A zero slope is the optimum of this stated training criterion. It can discard a real relationship if the selected penalty is too strong.

One gradient step shows what the residual signal does

Use $J(\beta) = \frac{1}{2n}\|X\beta - y\|^2$ and

$$g = \frac{1}{n} X^\top (X\beta - y), \qquad \beta_{\text{new}} = \beta - \eta g.$$

Starting at $b = w = 0$ on the running data gives

$$g = (-10/3, -23/3)^\top.$$

With $\eta = 0.1$, the new parameters are $(1/3, 23/30)$. Predictions rise toward the positive observations; J drops from $38/6 \approx 6.33$ to about 1.283.

Interpretation

The gradient aggregates feature-weighted prediction errors. Learning rate controls the step size; it does not choose the model's representational capacity.

A second gradient step uses the new residuals

After the first linear-regression update, $b = 1/3, w = 23/30$. Recompute the gradient rather than repeating its initial value.

$$g_b = -22/15, \quad g_w = -154/45; \qquad \eta = 0.1 \Rightarrow b_{\text{new}} = 12/25, \quad w_{\text{new}} = 499/450.$$

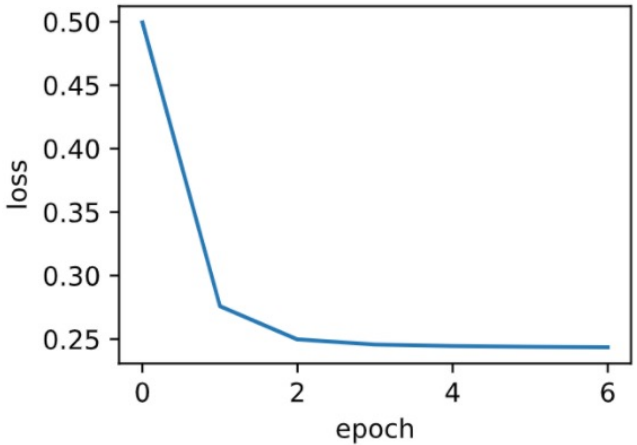
- The residuals before this update are $9/10, 17/15, 71/30$ under the true-minus-predicted convention.
- All remain positive, so both coefficients increase, but the gradient is smaller than at zero initialization.
- The algorithm follows current error information. Reusing the old gradient indefinitely would be a fixed straight move, not gradient descent on the changing objective.

Scaling and step size determine optimization behavior

For a quadratic least-squares objective, the Hessian is $H = X^\top X/n$. Full-batch gradient descent is stable with a step satisfying $0 < \eta < 2/\lambda_{\max}(H)$.

- Different feature units create unequal curvature; one shared step can overshoot a steep direction while barely moving in a flat one.
- Fit a scaler on training data, then transform held-out data using it.
- Plot objective against updates. Oscillation suggests an excessive step; a high flat curve can indicate tiny steps or poor scaling.
- Mini-batches reduce per-update work, but introduce gradient noise. Compare progress using both epochs and elapsed time.

Scaling and step size determine optimization behavior



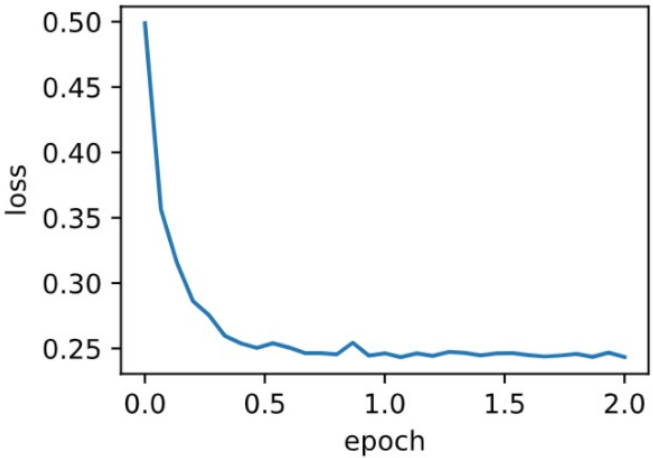
The running Hessian gives a numerical step-size check

For half-average squared loss on the running design, the Hessian is $X^\top X/3$.

$$H = \begin{bmatrix} 1 & 2 \\ 2 & 14/3 \end{bmatrix}, \quad \lambda_{\max}(H) = \frac{17 + \sqrt{265}}{6} \approx 5.55, \quad 0 < \eta < 2/\lambda_{\max} \approx 0.361.$$

- The demonstrated step 0.1 lies inside this full-batch quadratic stability range.
- The condition depends on the loss normalization and the actual feature matrix, not a universal recommended learning-rate number.
- Stochastic updates add further noise and require appropriate handling. A stable optimizer can still fit an unsuitable model or overfit the sample.

Reference convergence trace: the loss can fluctuate near a minimum



Residual structure reveals what the model has left unexplained

Residual pattern	Possible implication
Curved mean across fitted values	A useful nonlinear relationship was omitted
Spread increases with predicted value	Noise variance may change with input
Long runs of the same sign over time	Dependence, drift, or an omitted time component
One very large deviation	Check recording, measurement, and relevance

Coefficients describe conditional associations, not automatically causal effects. Under independent constant-variance errors and full rank, $\mathrm{Cov}(\hat{\beta}) = \sigma^2 (X^\top X)^{-1}$; weak directions increase uncertainty. Prediction assessment still requires held-out observations.

Compute several regression metrics from the same residuals

For $r = (1/6, -1/3, 1/6)$, $\mathrm{RSS} = 1/6$ and $n = 3$:

Metric	Calculation	Interpretation
MAE	$\sum \lvert r_i \rvert / 3 = 2/9$	Average absolute error, response units
MSE	$\mathrm{RSS}/3 = 1/18$	Large errors emphasized, squared units
RMSE	$\sqrt{1/18} \approx 0.236$	Same emphasis, response units
R^2	$1 - (1/6)/(14/3) = 27/28$	Relative to mean-response baseline

Here $\bar{y} = 10/3$ and $\mathrm{TSS} = \sum (y_i - \bar{y})^2 = 14/3$. These are training-set arithmetic examples. Their impressive R^2 does not establish performance on new students or future observations.

Response-unit changes scale error metrics differently

Multiply every response and prediction by ten, keeping the same relative fit.

$$\mathrm{MAE} \to 10\mathrm{MAE}, \quad \mathrm{MSE} \to 100\mathrm{MSE}, \quad \mathrm{RMSE} \to 10\mathrm{RMSE}, \quad R^2 \text{ unchanged.}$$

- The running MAE becomes $20/9$, MSE $100/18$, and RMSE about 2.357.
- R-squared is unchanged because both residual and total squared variation scale by one hundred.
- Report response units with absolute-error metrics. A numerically smaller loss from a different unit system is not evidence of better prediction, and relative metrics still depend on population difficulty.

The R-squared mean reference is a scoring benchmark

Training responses have mean $10/3$. On held-out responses $(2, 4)$, the test mean is three.

$$\hat{y} = (10/3, 10/3): \quad \mathrm{RSS} = 20/9, \quad \mathrm{TSS}_{\mathrm{test}} = 2, \quad R^2 = 1 - (20/9)/2 = -1/9.$$

- The fitted constant used only training labels, so its prediction is legitimate; the test mean appears in the score denominator, not as information given to the predictor.
- Predicting the test mean would give $R^2 = 0$ by definition, but is not a deployable baseline without knowing those future labels.
- Distinguish a metric's reference quantity from the information available during prediction.

Adjusted R-squared accounts for fitted columns within the sample

With n observations and p slope columns, the conventional adjusted coefficient of determination is

$$\bar{R}^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1}; \qquad n = 3, p = 1, R^2 = 27/28 \Rightarrow \bar{R}^2 = 13/14.$$

- The factor uses residual degrees of freedom; adding columns can raise raw training R-squared without sufficient error reduction.
- The expression requires a positive denominator. Tiny examples leave very few degrees of freedom.
- Adjusted R-squared remains an in-sample summary, not an independent test of the full feature-selection process or a guarantee of future prediction.

Metric choice can reverse a comparison

On three held-out cases, compare absolute errors:

Model	Absolute errors	MAE / RMSE
A	$(0, 0, 6)$	$2 \,/\, \sqrt{12} \approx 3.46$
B	$(2.5, 2.5, 2.5)$	$2.5 \,/\, 2.5$

MAE favors A; RMSE favors B because A has one costly large miss.

- For held-out $y = (0, 1)$ and predictions $(2, 2)$, RSS $= 5$, TSS $= 0.5$, so $R^2 = 1 - 5/0.5 = -9$. Negative means worse than the mean baseline.
- Percentage errors are unstable near zero; median error can hide tail mistakes.
- Report a meaningful baseline, subgroup behavior, and signed residual bias when relevant. Adjusted R^2 is an in-sample complexity correction, not a replacement for held-out evaluation.

A median error can hide a severe tail

Suppose absolute held-out errors are $(0, 0, 0, 0, 10)$.

$$\mathrm{MedAE} = 0, \qquad \mathrm{MAE} = 2, \qquad \mathrm{RMSE} = \sqrt{20} \approx 4.47.$$

- The median describes the middle case and is robust to the largest error; it also ignores its severity in this example.
- MAE includes the large miss linearly, while RMSE emphasizes it through squaring.
- Choose the primary view from the consequences of errors, and add a tail summary when a rare large miss matters. A zero median is not a statement that every prediction is exact.

Percentage error is undefined at a zero target

Three held-out targets are $(0, 1, 10)$ and each absolute prediction error is one.

$$\mathrm{APE}_i = |y_i - \hat{y}_i|/|y_i| : \quad 1/0 \text{ undefined}, \quad 1/1 = 100\%, \quad 1/10 = 10\%.$$

- The same absolute error has very different percentage interpretations near zero.
- Dropping zero rows silently changes the evaluated population. Adding an arbitrary small denominator can make the metric depend on that arbitrary choice.
- Use a task-appropriate absolute or scaled metric and state its reference. Do not select a percentage score merely because it looks comparable across response levels.

An overall average can conceal a weak subgroup

A held-out set has 90 cases from group A and ten from group B.

$$\mathrm{MAE}_A = 1, \quad \mathrm{MAE}_B = 10; \qquad \mathrm{MAE}_{\mathrm{all}} = \frac{90(1) + 10(10)}{100} = 1.9.$$

- The overall number is dominated by the common group and does not describe the typical B-case error.
- Equal group averaging would be 5.5, answering a different question than average error per observation.
- Report relevant group sizes with their metrics. Investigate whether the gap reflects omitted information, sampling, measurement, or a population shift before choosing a remedy.

Regression exercise: interpret the fitted line and its limits

Use the fitted line $\hat{y} = 1/3 + 1.5x$.

1. Predict at $x = 4$: $\hat{y} = 19/3$. Explain why this is extrapolation beyond the observed range.
2. Confirm RSS $= 1/6$ and identify the most overpredicted training observation.
3. If a second feature is an exact copy of x, explain why predictions can remain well defined while coefficients are nonunique.
4. Choose between MAE and RMSE when a single very large miss is especially costly.

Before reporting

A three-point training example teaches computation. Assess prediction with an appropriate held-out design and compare a useful baseline.

Solution: verify the regression arithmetic and identify the limitation

Use the fitted line $1/3 + 1.5x$ and the residuals already calculated.

$$x = 4 \Rightarrow \hat{y} = 19/3; \quad \mathrm{RSS} = 1/6; \quad \mathrm{MAE} = 2/9; \quad \mathrm{RMSE} = \sqrt{1/18}.$$

- The second training case is overpredicted because its residual is $-1/3$.
- A duplicated feature does not identify two separate effects; only their total slope matters.
- Input four is extrapolation. A mathematically valid formula can produce an answer outside the region where the data supported its relationship. Use a held-out design matched to the intended input range.

A sigmoid converts a linear score into an event probability

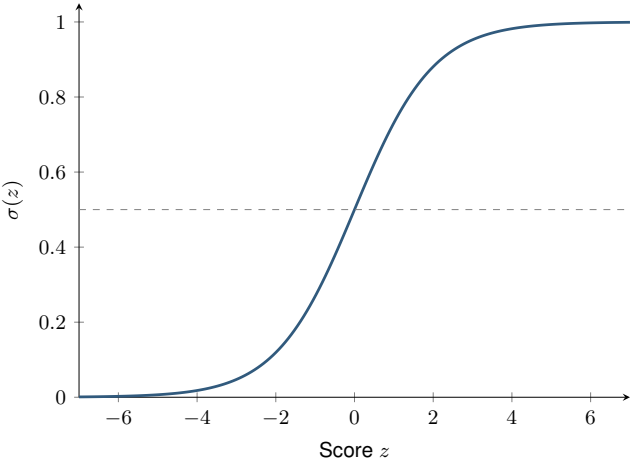
Use scaled inputs $x = (-1, 0, 1)$ and binary labels $y = (0, 0, 1)$.

$$z = b + wx, \qquad p(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

z	-2	-1	0	1	2
p	0.119	0.269	0.500	0.731	0.881

- The score is unrestricted; the probability is bounded. A threshold then converts probability into a class.
- $\sigma'(z) = p(1 - p)$ is largest at $p = 0.5$, where it equals 0.25.
- Equal score increments do not produce equal probability increments. Logistic regression models classification probabilities despite its name.

A sigmoid converts a linear score into an event probability



Invert the probability conversion to check a score

The sigmoid and logit are inverse transformations on probability values strictly between zero and one.

$$z = \log 2 \Rightarrow p = 2/3; \quad z = -\log 3 \Rightarrow p = 1/4; \quad \log \frac{p}{1-p} = z.$$

- At $p = 2/3$, odds are two, so the recovered score is $\log 2$.
- The derivative $p(1-p)$ is $2/9$ at probability $2/3$ and $3/16$ at $1/4$.
- Finite scores never give probability exactly zero or one mathematically. Numerical software may round extreme values, so use stable likelihood computations rather than taking logarithms of rounded outputs.

Log odds are linear; the probability is not

$$\frac{p}{1-p} = e^{z}, \qquad \log \frac{p}{1-p} = z = b + w^{\top} x.$$

Coefficient meaning

Increasing x_j by one while holding other features fixed multiplies the odds by e^{w_j}. If $w_j = \log 2$, odds double.

- At $p = 0.5$, odds are 1; doubling them gives $p = 2/3$.
- At $p = 0.2$, odds are $1/4$; doubling them gives $p = 1/3$.
- With threshold 0.5, the decision boundary is $b + w^{\top} x = 0$. At threshold t, it is $b + w^{\top} x = \log(t/(1-t))$.

An odds multiplier produces different probability increments

Let a coefficient be $w = \log 2$ and hold other modeled features fixed.

$$p_0 = 0.2 \Rightarrow \mathrm{odds}_0 = 1/4; \quad \Delta x = 1: \ p_1 = 1/3; \quad \Delta x = 2: \ p_2 = 1/2.$$

- One input unit doubles odds; two units multiply odds by four.
- Probability rises first by about 0.133, then by about 0.167, although each score increase is the same.
- A reported odds ratio needs a stated input unit and reference. This is a conditional fitted association, not automatically the effect of a real intervention on that input.

Bernoulli likelihood explains cross entropy and confident errors

For $y_i \in \{0, 1\}$ and $p_i = \sigma(b + w^\top x_i)$,

$$p(y_i \mid x_i) = p_i^{y_i}(1 - p_i)^{1-y_i}.$$

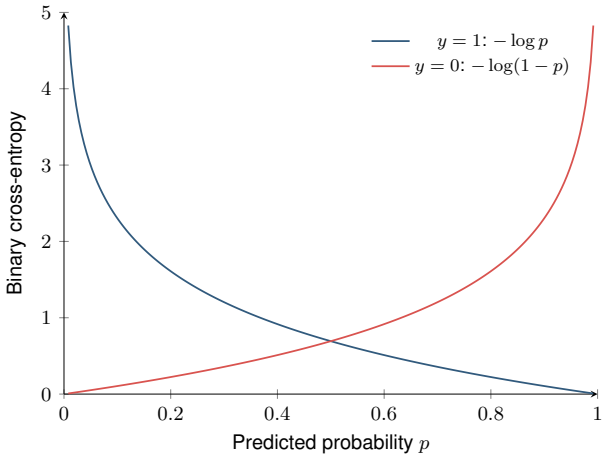
Independent observations give average negative log likelihood

$$J = -\frac{1}{n}\sum_i [y_i \log p_i + (1 - y_i)\log(1 - p_i)].$$

True label	Predicted positive probability	True-label probability	Loss
1	0.9	0.9	0.105
1	0.01	0.01	4.605
0	0.9	0.1	2.303

Natural logarithms are used. This smooth objective rewards probability assigned to the observed label. Zero–one error counts wrong classes equally; cross entropy penalizes confident mistakes more strongly.

Bernoulli likelihood explains cross entropy and confident errors



Compare complete-data likelihood before and after an update

Labels are $(0, 0, 1)$. Initial positive-class probabilities are all 0.5. After one update: $p \approx (0.378, 0.458, 0.542)$.

$$p(D)_{\text{initial}} = 0.5^3 = 0.125,$$
$$p(D)_{\text{new}} \approx 0.622(0.542)(0.542) \approx 0.183.$$

- The negative examples contribute one minus their positive probabilities; the positive contributes its positive probability.
- Average negative log likelihood falls from about 0.693 to 0.567, consistent with the larger joint likelihood.
- Independence justifies multiplying observation probabilities. The likelihood compares parameter settings on observed labels; independent prediction still requires separate assessment.

The chain rule reduces the gradient to probability minus label

For one observation, $\ell = -y \log p - (1-y)\log(1-p)$ and $p = \sigma(z)$:

$$\frac{\partial \ell}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p}, \qquad \frac{\partial p}{\partial z} = p(1-p).$$

Multiplying and simplifying yields $\partial \ell / \partial z = p - y$.

$$\nabla_w J = \frac{1}{n}\sum_i (p_i - y_i) x_i, \qquad \frac{\partial J}{\partial b} = \frac{1}{n}\sum_i (p_i - y_i).$$

Direction

For a positive observation with too-small probability, $p - y < 0$; moving against the gradient increases its score. The compact formula follows from the model and loss together.

The logistic residual directs a correction for each label

For one feature value $x = 2$, compare two confidently mistaken binary observations.

$$y = 1, p = .2: \ \frac{\partial \ell}{\partial w} = (.2 - 1)2 = -1.6; \qquad y = 0, p = .8: \ \frac{\partial \ell}{\partial w} = 1.6.$$

- Gradient descent increases the weight in the first case and decreases it in the second.
- The intercept derivative is $p - y$ without the feature multiplier; negative feature values can reverse the weight direction.
- A batch update averages all observations, so competing corrections share the same parameters. The gradient formula follows from both sigmoid and cross entropy, not from the class threshold.

Compute one logistic gradient update on three observations

For $x = (-1, 0, 1)$, $y = (0, 0, 1)$, start with $b = w = 0$. Then every $p_i = 0.5$.

$$g_b = (0.5 + 0.5 - 0.5)/3 = 1/6,$$

$$g_w = \big[(-1)(0.5) + 0(0.5) + 1(-0.5)\big]/3 = -1/3.$$

With $\eta = 1$, obtain $b = -1/6$, $w = 1/3$, scores $(-1/2, -1/6, 1/6)$ and probabilities approximately $(0.378, 0.458, 0.542)$.

Check

The two negatives move below 0.5, the positive moves above it; average cross entropy falls from 0.693 to about 0.567. This is one step, not the final optimum.

A second logistic update recomputes the probability residuals

After the first step, use $b = -1/6$, $w = 1/3$ and probabilities approximately $0.378, 0.458, 0.542$.

$$g_b \approx 0.126, \quad g_w \approx -0.279; \qquad \eta = 1 \Rightarrow b_{\text{new}} \approx -0.293, \quad w_{\text{new}} \approx 0.612.$$

- New positive probabilities are about $(0.288, 0.427, 0.579)$: negatives decrease further and the positive increases.
- The update uses current probabilities; repeating the first gradient would ignore the changed sigmoid response.
- Parameter improvement should be checked through the objective and suitable stopping criteria. These separable training observations also illustrate why unregularized coefficient growth needs attention.

Convexity supports optimization, but uniqueness needs conditions

With an intercept column included in X, the average-loss Hessian is

$$H = \frac{1}{n} X^\top S X, \qquad S = \mathrm{diag}\{p_i(1-p_i)\}.$$

For any v, $v^\top H v = \frac{1}{n}\sum_i p_i(1-p_i)(x_i^\top v)^2 \geq 0$.

- Logistic cross entropy is convex in linear-model coefficients.
- Positive definiteness requires further rank and probability conditions; semidefiniteness alone does not imply a unique finite optimum.
- Newton methods solve $H\Delta = -g$ and update parameters using local curvature. Regularization and numerical solvers help weak directions.

Newton can solve an intercept-only logistic model quickly

Ignore features temporarily and fit one event probability to labels $(0, 0, 1)$.

$$b = 0: \; g = 1/6, \; H = 1/4; \qquad \Delta b = -g/H = -2/3, \quad p_{\text{new}} \approx 0.339.$$

- The exact unregularized optimum has $p = 1/3$ and intercept $\log(1/2) \approx -0.693$.
- The Newton step uses curvature, moving close to that optimum in this simple problem; it is not the same as taking learning rate one on the gradient.
- Larger feature systems require solving a Hessian-based linear system and handling rank or separation issues. One successful small step does not guarantee all numerical cases are easy.

Complete separation can push unregularized coefficients outward

Suppose a score z_i is positive for every positive label and negative for every negative label. Scale all scores by $c > 0$.

- As $c \to \infty$, probabilities of the observed labels approach 1.
- Cross entropy approaches 0, while coefficient magnitudes can grow without a finite minimizer.
- Adding $\lambda \|w\|^2/2$ penalizes this growth and yields a controlled tradeoff in the slope directions.
- Software's C usually represents inverse regularization strength: larger C means weaker regularization, within its stated objective convention.

Regularization is valuable for stability; a finite unregularized estimate can still exist in nonseparable, identifiable data.

Scaling a separating score keeps labels but lowers cross entropy

The first updated scores $(-1/2, -1/6, 1/6)$ separate labels $(0, 0, 1)$. Multiply them by six.

$$z_{\text{scaled}} = (-3, -1, 1), \qquad p \approx (0.047, 0.269, 0.731), \qquad J \approx 0.225.$$

- The threshold-0.5 labels remain unchanged while average cross entropy drops from about 0.567.
- Further outward scaling can keep reducing unregularized training loss toward zero without a finite coefficient optimum.
- A coefficient penalty makes this growth costly. Correct training classes, finite optimization, and calibrated future probabilities are separate concerns.

Interpret a coefficient with a reference, a unit, and a model

- e^{w_j} is an odds multiplier for one unit of x_j, holding modeled covariates fixed.
- If x_j is standardized, one unit means one training-set standard deviation; if categorical, identify the reference category.
- A coefficient is a conditional association within the fitted model, not proof of a causal intervention effect.
- Class weighting changes the fitted objective; its raw probability can reflect a reweighted population rather than the original prevalence.

Practical consequence

Assess calibration and decision thresholds on validation data with the intended population proportions. Do not interpret a weighted model's 0.8 output as calibrated solely because it is a valid probability.

Decision costs determine a probability threshold

Let false-positive cost be C_{FP} and false-negative cost be C_{FN}, with zero cost for correct decisions.

$$R(\text{predict } 1 \mid x) = C_{\mathrm{FP}}(1 - p), \qquad R(\text{predict } 0 \mid x) = C_{\mathrm{FN}}p.$$

Choose positive when

$$p > \frac{C_{\mathrm{FP}}}{C_{\mathrm{FP}} + C_{\mathrm{FN}}}.$$

If a miss costs four times a false alarm, the ideal threshold is $1/(1 + 4) = 0.2$. This derivation assumes probabilities appropriate to the target population and the stated cost model. Operational limits and imperfect calibration require validation-based threshold assessment.

Expected costs can be calculated for two individual probabilities

Use false-alarm cost one and missed-positive cost four, giving threshold 0.2.

$$p = .1 : \ R(1) = .9, \ R(0) = .4; \qquad p = .3 : \ R(1) = .7, \ R(0) = 1.2.$$

- At probability 0.1, negative has lower expected cost; at 0.3, positive has lower expected cost.
- This is a post-model decision comparison. It does not require refitting the coefficients whenever the cost changes.
- The probabilities must be appropriate to the intended population, and the cost model must reflect the actual consequences. Resource constraints can add a further requirement beyond this simple threshold.

Softmax jointly normalizes classes; one-versus-rest fits them separately

For class scores $z_c = w_c^\top x + b_c$,

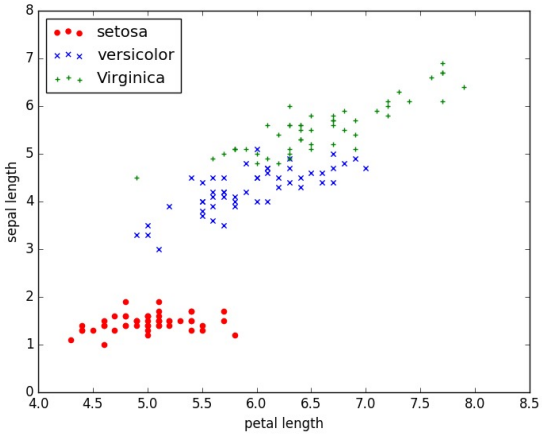
$$p_c = \frac{e^{z_c}}{\sum_k e^{z_k}}, \qquad \ell = -\sum_c y_c \log p_c = -\log p_{\text{true}}.$$

Calculation

Scores $(0, \log 2, \log 3)$ give probabilities $(1/6, 1/3, 1/2)$. If the true class is 3, loss is $\log 2 \approx 0.693$.

- Softmax fits one joint multiclass likelihood; its probabilities sum to one.
- One-versus-rest fits one binary classifier for each class against all others. Its separately fitted outputs require a stated combination rule.
- Subtract the largest score before exponentiation for numerical stability. A shared score shift leaves softmax probabilities unchanged.

Iris: three classes represented by two measured features



Softmax gradients compare every class probability with its label

Use class probabilities $(1/6, 1/3, 1/2)$ and a true class-three one-hot vector $(0, 0, 1)$.

$$\frac{\partial \ell}{\partial z_c} = p_c - y_c : \qquad g_z = (1/6, 1/3, -1/2).$$

- A negative gradient step raises the true-class score and lowers the others, with effects coupled by the shared softmax denominator.
- For a feature coefficient, the same score gradient is multiplied by that feature value.
- Adding the same constant to all scores leaves probabilities unchanged. Subtracting the maximum is therefore a stable computation trick, not a new predictive assumption.

A pipeline keeps fitting and validation in the right order

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

pipe = make_pipeline(StandardScaler(),
    LogisticRegression(max_iter=2000))
search = GridSearchCV(pipe,
    {"logisticregression__C": [0.1, 1, 10]},
    cv=5, scoring="neg_log_loss")
search.fit(X_dev, y_dev)
p_test = search.predict_proba(X_test)[:, 1]
```

Binary labels must be ordered consistently with the probability column. Use time/group-aware folds when needed. Negative log-loss scoring is maximized, so less negative means better. Threshold selection uses development validation predictions, before final test assessment.

Period 2 exercise: connect score, probability, loss, and action

For $b = -1/6$, $w = 1/3$ and $x = 1$, use the result from one gradient update.

1. Compute $z = 1/6$ and $p \approx 0.542$.
2. If $y = 1$, compute the cross-entropy contribution $-\log p \approx 0.613$.
3. Give the reported class at thresholds 0.5 and 0.6. Explain why the probability model is unchanged.
4. If the same sample remains separable, explain what can happen to unregularized coefficients during further fitting.

Answer structure

A linear score produces a probability; likelihood fits the coefficients; a chosen operating threshold produces a decision. Next, evaluate that decision using the errors it creates.

Solution: probability, class, and cost give three different answers

At the first-step parameters and input one, the probability is about 0.542.

$$\ell(y = 1) = -\log(.542) \approx .613; \quad C_{\mathrm{FP}} = 1, C_{\mathrm{FN}} = 4 : \; R(1) = .458 < R(0) = 2.168.$$

- Threshold 0.5 reports positive; threshold 0.6 reports negative. The same probability underlies both decisions.
- Under the stated cost model, the positive decision has lower expected cost if the probability is appropriate.
- Training updates, threshold selection, and final assessment are separate stages. A changed threshold should be chosen on validation outcomes and frozen before test reporting.

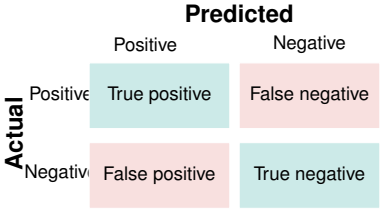
One confusion matrix supports several different questions

On 1,000 independent held-out inspection records, 50 truly need follow-up. Positive means "needs follow-up"; rows are true labels.

	Predict positive	Predict negative
True positive	TP $= 40$	FN $= 10$
True negative	FP $= 95$	TN $= 855$

- Accuracy $= (40 + 855)/1000 = 0.895$.
- Always predicting negative scores $950/1000 = 0.95$ but finds none of the positives.
- The positive class and table orientation must be stated; otherwise precision and recall denominators are easy to reverse.

One confusion matrix supports several different questions



Precision reads a prediction column; recall reads an actual row.

The majority baseline can win accuracy and lose the decision task

On the running 1,000 cases, compare always-negative prediction with the fitted operating point.

Always negative : $\text{Accuracy} = .95,\ \text{cost} = 20(50) = 1000;$ model : $.895,\ \text{cost} = 295.$

- The model's lower accuracy comes with finding 40 of the 50 positives; the baseline finds none.
- Accuracy treats each record equally, while the declared cost makes missed positives especially important.
- A baseline remains essential, but which baseline score matters follows the task. Report both the counts and the decision consequences rather than choosing whichever single metric favors the preferred method.

Compute precision, recall, specificity, and F1 from those counts

Question	Formula	Running value
Are alerts credible?	Precision $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$	$40/135 = 0.296$
How many positives found?	Recall $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$	$40/50 = 0.8$
How many negatives cleared?	Specificity $= \mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$	$855/950 = 0.9$
Balance precision and recall	$F_1 = 2\mathrm{TP}/(2\mathrm{TP}+\mathrm{FP}+\mathrm{FN})$	$80/185 = 0.432$

Recall conditions on true positives; precision conditions on predicted alerts. Balanced accuracy averages recall and specificity: $(0.8 + 0.9)/2 = 0.85$. None of these values alone specifies the cost of mistakes.

F-beta changes the emphasis but does not directly encode a cost matrix

Use TP 40, FP 95, and FN 10 from the same confusion matrix.

$$F_\beta = \frac{(1+\beta^2)\mathrm{TP}}{(1+\beta^2)\mathrm{TP} + \beta^2\mathrm{FN} + \mathrm{FP}}; \quad F_2 = 200/335 \approx .597.$$

- For comparison, F1 is $80/185 \approx 0.432$. Beta two increases the influence of recall-related omissions.
- The square of beta appears in the formula; it is not simply an arithmetic weight of two.
- F-beta is a summary of positive-class retrieval. It does not count true negatives or automatically reproduce a specified false-positive/false-negative financial cost.

Build a confusion matrix from a short label list

True labels are $(1, 0, 1, 0, 1)$ and predicted labels are $(1, 1, 0, 0, 1)$.

$$\mathrm{TP} = 2, \quad \mathrm{FP} = 1, \quad \mathrm{FN} = 1, \quad \mathrm{TN} = 1.$$

- Accuracy is $3/5 = 0.6$, precision $2/3$, recall $2/3$, and specificity $1/2$.
- The second case is a false positive; the third is a false negative. Name the true and predicted labels before choosing the cell.
- Each observation enters exactly one cell. The four counts must sum to five; true-positive-row counts must sum to the three actual positives.

Moving the threshold changes the counts, not the fitted scores

Illustrative results on the same 50 positives and 950 negatives:

Threshold	TP	FP	FN	TN	Cost $20\mathrm{FN} + \mathrm{FP}$
0.8	25	19	25	931	519
0.5	40	95	10	855	295
0.2	45	190	5	760	290

Lower thresholds find more positives while creating more false alarms. Here the lowest listed cost is at 0.2, but only three candidate thresholds were evaluated. Select an operating threshold on validation data, consider follow-up capacity, and then freeze it before independent test reporting.

Changing the error cost can reverse the threshold choice

Reuse the same threshold counts, but now a false alarm costs five and a missed positive costs one.

$$t = .8 : 25 + 5(19) = 120; \quad t = .5 : 10 + 5(95) = 485; \quad t = .2 : 5 + 5(190) = 955.$$

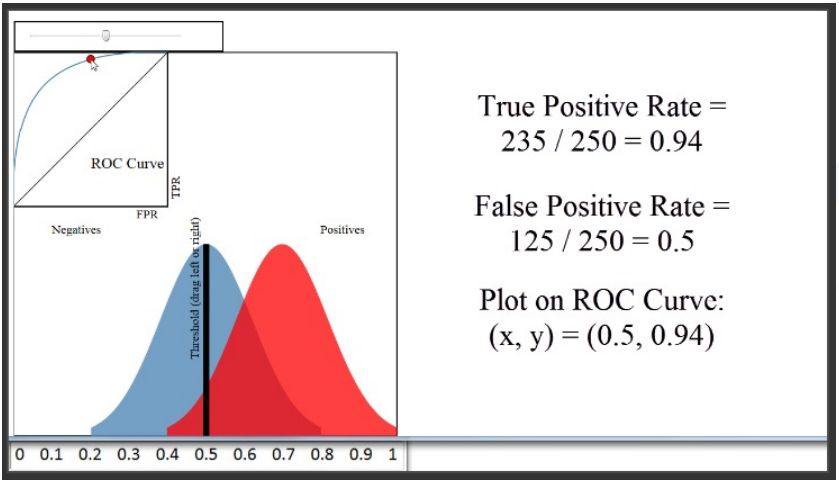
- Among these listed candidates, threshold 0.8 now has the lowest cost, whereas the earlier miss-heavy model favored a lower threshold.
- The probability scores and true labels did not change; the decision objective did.
- The chosen operating policy should reflect actual consequences and resource limits. Use validation to select it, then report independent counts rather than retrofitting the cost story to a test result.

ROC and PR plot different views of the same threshold sweep

Threshold	FPR $= \mathrm{FP}/950$	Recall $= \mathrm{TP}/50$	Precision
0.8	0.02	0.50	$25/44 = 0.568$
0.5	0.10	0.80	$40/135 = 0.296$
0.2	0.20	0.90	$45/235 = 0.191$

- ROC uses FPR horizontally and recall vertically: discrimination relative to negatives.
- PR uses recall horizontally and precision vertically: coverage versus alert credibility.
- With rare positives, a small FPR can still produce many false alerts. Inspect the actual counts and PR behavior.
- Curves compare rankings across thresholds; one operating point still has to be selected.

A threshold maps score distributions to one ROC point



True Positive Rate =
235 / 250 = 0.94

False Positive Rate =
125 / 250 = 0.5

Plot on ROC Curve:
(x, y) = (0.5, 0.94)

The extreme thresholds clarify ROC and PR endpoints

On 50 positives and 950 negatives, imagine predicting none positive or all positive.

$$t \text{ very high}: \ (\mathrm{TPR}, \mathrm{FPR}) = (0, 0); \quad t \text{ very low}: \ (1, 1), \ \mathrm{Precision} = 50/1000 = .05.$$

- With no alerts, precision has denominator zero and is mathematically undefined; plotting libraries may add a convention for display.
- With all alerts, recall is one but precision equals prevalence, because every negative is also reported.
- These endpoints explain why a high-recall point is not enough. Curves summarize a threshold sweep, while the selected point must still have useful counts and consequences.

AUC and average precision summarize ranking in different ways

Ten distinct scores are sorted high to low. Their true labels are

$$(1, 0, 1, 0, 0, 1, 0, 0, 0, 0).$$

- Three positives outrank $7, 6, 4$ negatives respectively. ROC AUC $= (7 + 6 + 4)/(3 \cdot 7) = 17/21 \approx 0.810$.
- Positives occur at ranks 1, 3, 6, where precision is $1, 2/3, 3/6$. Average precision $= (1 + 2/3 + 3/6)/3 = 13/18 \approx 0.722$.
- AUC measures positive–negative ordering; ties contribute one half under the usual convention.
- Random-ranking PR precision baseline equals prevalence, here $3/10 = 0.3$. These are ranking summaries, not calibration measures or operating costs.

The same ranking can have different probability loss

Use true labels $(1, 0, 1, 0)$ and probabilities $(0.8, 0.6, 0.4, 0.2)$. Then replace each probability by its square.

$$\mathrm{AUC} = 3/4 \text{ for both;} \qquad \mathrm{LogLoss}_{p} \approx 0.570, \quad \mathrm{LogLoss}_{p^2} \approx 0.6915.$$

- Squaring is strictly increasing on this range, so all positive–negative score orders remain unchanged.
- The true-label probabilities change, and in this example their average negative log probability worsens.
- Good ranking does not establish calibration. Evaluate probabilities separately when they determine risk communication or cost-derived decisions; monotone score transformations are not automatically harmless.

Evaluate probabilities as well as thresholded labels

$$\mathrm{LogLoss} = -\frac{1}{n}\sum_i [y_i \log p_i + (1-y_i)\log(1-p_i)], \quad \mathrm{Brier} = \frac{1}{n}\sum_i (p_i - y_i)^2.$$

- Log loss strongly penalizes confidently wrong probabilities.
- Brier measures squared probability error on a bounded scale.
- Calibration asks whether objects assigned about 0.8 risk experience the event about 80% of the time.
- Ranking, calibration, and one threshold's confusion matrix are distinct views. A monotone score transform preserves rank but can damage calibration.

Assessment

Use held-out probabilities from the intended population; show sufficient counts when comparing calibration groups. A small bin is not a reliable frequency estimate.

Macro and micro averages emphasize different classes

Rows are true labels; columns are predicted labels:

	A	B	C
A	90	8	2
B	3	12	5
C	1	4	5

Class recalls are $0.90, 0.60, 0.50$. Macro recall $= (0.9 + 0.6 + 0.5)/3 = 0.667$; micro recall $= 107/130 = 0.823$ for this single-label task.

- Macro treats each class equally; micro pools observation counts.
- Weighted averages reflect class supports and can conceal poor minority performance.
- Examine specific off-diagonal substitutions; a total score does not show which categories are confused.

Weighted recall matches pooled accuracy in this single-label example

For the earlier three-class matrix, class supports are $100, 20, 10$ and recalls are $0.9, 0.6, 0.5$.

$$\mathrm{WeightedRecall} = \frac{100(.9) + 20(.6) + 10(.5)}{130} = 107/130 \approx .823; \quad \mathrm{MacroRecall} = 2/3.$$

- Weighting by class support reproduces the pooled correct-count fraction for single-label classification.
- Equal class weighting gives the minority class the same contribution as the largest class, but its estimate may be noisier because it has fewer cases.
- Report supports with the chosen average. The weighting scheme reflects the evaluation question; it cannot repair poor performance on a class that matters.

MCC uses all four cells, but still needs context

$$\mathrm{MCC} = \frac{\mathrm{TP}\,\mathrm{TN} - \mathrm{FP}\,\mathrm{FN}}{\sqrt{(\mathrm{TP}+\mathrm{FP})(\mathrm{TP}+\mathrm{FN})(\mathrm{TN}+\mathrm{FP})(\mathrm{TN}+\mathrm{FN})}}.$$

For $\mathrm{TP} = 40$, $\mathrm{TN} = 855$, $\mathrm{FP} = 95$, $\mathrm{FN} = 10$:

$$\mathrm{MCC} = \frac{33250}{\sqrt{135 \cdot 50 \cdot 950 \cdot 865}} \approx 0.446.$$

- MCC is a correlation-type summary ranging from -1 to 1 when defined.
- It includes both positive and negative outcomes, unlike F1.
- Degenerate prediction tables need an explicit software convention; a single aggregate still does not replace counts or a cost-sensitive operating point.

Class weighting changes the importance of observations

For the 50 positives and 950 negatives, balanced weights are

$$\alpha_1 = \frac{1000}{2 \cdot 50} = 10, \qquad \alpha_0 = \frac{1000}{2 \cdot 950} = \frac{10}{19}.$$

Each class contributes total weight 500.

$$J = -\frac{1}{n} \sum_i \alpha_{y_i} [y_i \log p_i + (1 - y_i) \log(1 - p_i)] + \text{penalty}.$$

- Weighting encourages attention to minority errors without creating new observations.
- It can change coefficients and probability calibration; it is different from moving a fixed model's threshold.
- Evaluate probabilities and decisions on validation data with target prevalence. Balanced weighting is a candidate, not a universal cure.

Weighted training can represent a different conditional probability

In an ideal weighted Bernoulli fit, original event probability p is transformed by class weights α_1, α_0.

$$q = \frac{\alpha_1 p}{\alpha_1 p + \alpha_0 (1-p)}; \quad p = .05,\ \alpha_1 = 10,\ \alpha_0 = 10/19 \Rightarrow q = .5.$$

- The balanced-weight output 0.5 can correspond to only 5% prevalence in this illustrative no-feature case.
- This relation assumes an ideal model for the reweighted population; a finite misspecified fit need not obey it exactly.
- Evaluate calibration on target-distribution validation data. Moving a threshold and changing training weights are different operations with different probability interpretations.

Resampling changes training data, with specific tradeoffs

Method	What changes and what can go wrong
Oversampling	Repeat minority observations; no new independent evidence, possible overfitting
Undersampling	Remove majority observations; lower cost but lose information
SMOTE	Interpolate nearby minority inputs; locality may be wrong near overlap

$$x_{\text{new}} = (1 - t)x_i + tx_j, \quad 0 < t < 1.$$

For $x_i = (2, 4), x_j = (6, 8), t = 0.25$, the synthetic input is $(3, 5)$. It is not an observed person. Apply resampling only inside training folds; validation and test retain the intended natural distribution.

A minority interpolation can cross an inappropriate region

Two minority inputs are $(0, 0)$ and $(2, 2)$, but the middle neighborhood around $(1, 1)$ belongs predominantly to the majority class.

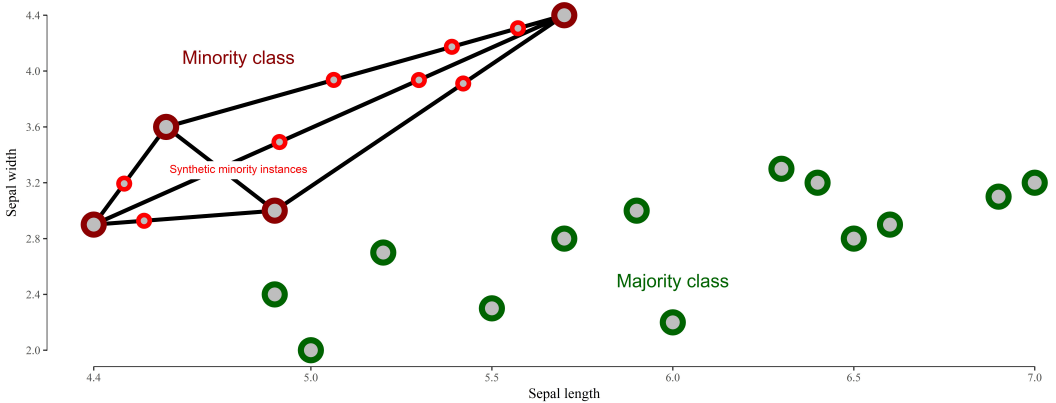
$$x_{\text{new}} = \tfrac{1}{2}(0, 0) + \tfrac{1}{2}(2, 2) = (1, 1).$$

- SMOTE would assign the minority label to this synthetic midpoint, even though the local class structure contradicts it.
- Nearby minority endpoints do not guarantee that their connecting segment remains inside the minority region.
- Inspect representation, neighborhood scale, categorical constraints, and overlap. Perform any synthesis only in training folds and evaluate on natural-distribution observations rather than synthetic test points.

SMOTE interpolates between neighboring minority observations

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Addressing class imbalance problems of ML via SMOTE: synthesising new dots between existing dots



@rikunert

Prevalence changes precision even when error rates stay fixed

Let prevalence be $\pi = P(Y = 1)$, recall be TPR, and false-positive rate be FPR. Then

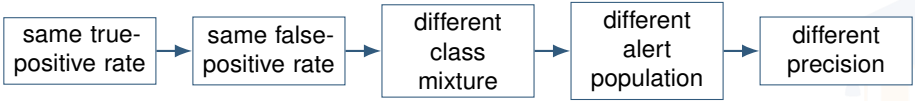
$$\text{Precision} = \frac{\text{TPR}\,\pi}{\text{TPR}\,\pi + \text{FPR}(1-\pi)}.$$

With TPR $= 0.8$ and FPR $= 0.1$:

Population	Positive prevalence	Precision
Artificially balanced	0.50	$0.4/(0.4 + 0.05) = 0.889$
Natural rare-positive	0.05	$0.04/(0.04 + 0.095) = 0.296$

Evaluating precision on a balanced test sample can overstate alert credibility in the real population. When population composition changes, reassess probabilities and operating performance.

Prevalence changes precision even when error rates stay fixed



A pure prevalence shift changes posterior odds under strict assumptions

Assume class-conditional feature distributions stay fixed while prevalence changes from 0.5 to 0.05.

$$\text{odds}_{\text{new}} = \text{odds}_{\text{old}} \frac{.05/.95}{.5/.5}; \quad p_{\text{old}} = .8 \Rightarrow \text{odds}_{\text{new}} = 4/19, \quad p_{\text{new}} = 4/23 \approx .174.$$

- The feature evidence remains the same, but the prior event frequency changes the posterior probability.
- This adjustment assumes a pure prior shift; changed measurement distributions or label definitions require a broader reassessment.
- A probability learned from artificially balanced data should not automatically be reported as natural-population risk. Check the sampling scheme and use appropriate target validation.

An imbalanced evaluation must preserve independent evidence

1. Define the future population and the positive event.
2. Split independent units; use groups or time when records are related.
3. Inside each training fold, fit scaling and any resampling or weighting procedure.
4. Compare candidates using the selected validation metric; assess thresholds on untouched validation outcomes.
5. Freeze the full procedure. On the test set, report prevalence, counts, probability performance, and the chosen operating point.

Stratification

Preserve class proportions when independent random splitting is appropriate. It cannot create more minority information or resolve patient/time dependence. Few positive test events imply substantial uncertainty.

The number of positive events governs recall uncertainty

Two independent evaluation sets both report recall 0.8, but one contains ten positives and the other fifty.

$$\mathrm{SE}(\widehat{\mathrm{Recall}}) \approx \sqrt{.8(.2)/n_+}: \quad n_+ = 10 \Rightarrow .126, \quad n_+ = 50 \Rightarrow .0566.$$

- The same displayed recall can have different sampling precision; one missed case changes ten-positive recall by ten percentage points.
- A large total count of negatives does not replace independent positive events for estimating recall.
- These are approximate standard errors, requiring appropriate independent sampling. Report positive counts and uncertainty when small differences influence selection or decisions.

Choose evaluation from the question and the costly error

Use	Primary view	Necessary companion
Predict a numerical response	MAE or RMSE	Baseline and residual structure
Rank rare candidate events	PR/AP or top-list yield	Prevalence and reviewed count
Communicate event probability	Log loss/Brier and calibration	Counts and target population
Make a costly binary action	Expected cost at selected threshold	Confusion matrix and capacity
Treat all categories fairly	Macro/class-specific metrics	Supports and confusion matrix

Define the metric before comparison. A model can be good at one use and poor at another; state what the selected evidence supports.

Period 3 exercise: choose a threshold under a resource limit

Use the earlier threshold table. A missed positive costs 20 units, a false alert costs 1, and at most 150 alerts can be reviewed.

1. At $t = 0.8$, there are $25 + 19 = 44$ alerts and cost 519.
2. At $t = 0.5$, there are $40 + 95 = 135$ alerts and cost 295.
3. At $t = 0.2$, there are $45 + 190 = 235$ alerts and cost 290.

Which listed threshold is feasible and has lowest cost? Explain why maximizing recall alone is insufficient. State where this operating choice must be made before reporting final performance.

Solution: choose a feasible policy and preserve the evaluation boundary

Under missed-positive cost 20, false-alarm cost one, and capacity 150 alerts, revisit the three candidate thresholds.

$$t = .8 : (44, 519), \quad t = .5 : (135, 295), \quad t = .2 : (235, 290) \quad \text{(alerts, cost)}.$$

- Threshold 0.2 is infeasible despite the smallest listed cost. Among feasible listed candidates, 0.5 has lower cost than 0.8.
- This does not prove global optimality over every threshold; finer validation comparisons can investigate the remaining range.
- Freeze the chosen probability model, preprocessing, and threshold before test assessment. Report its actual counts and capacity requirement rather than only a summary score.

Connect the model, its objective, and its evidence

Stage	Linear regression	Logistic regression
Prediction	$b + w^\top x$	$\sigma(b + w^\top x)$
Probability story	Gaussian response noise	Bernoulli event outcome
Fit objective	Squared residuals, optional penalty	Cross entropy, optional penalty
Selection	Features, penalty, solver settings	Penalty, weighting, threshold
Assessment	Held-out error and residuals	Held-out probabilities, counts, costs

Bridge to SVM

The next lecture keeps the same linear score $b + w^\top x$, but chooses it through margins and hinge loss. Compare the learning objective before comparing algorithm names.